



TẬP ĐOÀN BƯU CHÍNH VIỄN THÔNG VIỆT NAM
VNPT IT - TRUNG TÂM DMS
Bộ phận Phát triển Giải pháp AI

BÁO CÁO KIỂM THỬ HIỆU NĂNG HỆ THỐNG HỎI ĐÁP DỮ LIỆU TEXT-TO-SQL DƯỚI TẢI NGƯỜI DÙNG ĐỒNG THỜI

Load Test bằng công cụ k6 — Xác định ngưỡng bão hoà
trên nền tảng Vanna, Ollama llama3.2 và PostgreSQL Neon

PHẠM VI KIỂM THỬ

Báo cáo trình bày kết quả kiểm thử tải (load test) endpoint hỏi đáp AI /api/vanna/v2/chat_sse của hệ thống PoC Text-to-SQL. Mục tiêu là mô phỏng số lượng người dùng đồng thời tăng dần (5, 15, 30 và 50 người dùng), đo thời gian phản hồi, thông lượng và tỉ lệ lỗi, từ đó xác định điểm bão hoà và đề xuất phương án nâng cao khả năng chịu tải.

Tóm tắt thông số kiểm thử

Công cụ kiểm thử:	k6 v2.1.0, kịch bản ramping-vus
Đối tượng:	Endpoint SSE của Vanna 2.0 (FastAPI, Uvicorn 1 worker)
Hạ tầng AI:	Ollama llama3.2 cục bộ, PostgreSQL trên Neon
Mức tải:	5 → 15 → 30 người dùng và cao điểm 50 người dùng
Kết quả chính:	Ngưỡng bão hoà $\approx$ 5–10 người dùng đồng thời

Người thực hiện:
Lê Đức Anh
Trung tâm DMS

Mục lục

1	Tóm tắt kết quả cho Lãnh đạo (Executive Summary)	2
2	Môi trường và phương pháp kiểm thử	2
3	Kết quả kiểm thử	4
4	Phân tích kết quả	6
5	Đánh giá theo tiêu chí nghiệm thu (SLA)	7
6	Giới hạn của kiểm thử	8
7	Kết luận	8
8	Khuyến nghị cải thiện	8
	Phụ lục A. Kịch bản k6	10
	Tài liệu tham khảo	11

1. Tóm tắt kết quả cho Lãnh đạo (Executive Summary)

Báo cáo đánh giá khả năng chịu tải của hệ thống hỏi đáp dữ liệu bằng ngôn ngữ tự nhiên (Text-to-SQL) khi số lượng người dùng truy cập đồng thời tăng cao. Công cụ **k6** được dùng để phát sinh tải thực tế lên endpoint streaming `/api/vanna/v2/chat_sse`, mô phỏng các câu hỏi nghiệp vụ tiếng Việt.

Kết quả cho thấy hệ thống PoC vận hành **ổn định dưới 10 người dùng đồng thời**: thời gian phản hồi nhanh, gần như không có lỗi. Tuy nhiên, khi vượt ngưỡng này, hệ thống **bão hoà**: thông lượng không tăng thêm mà độ trễ và tỉ lệ lỗi tăng phi tuyến. Ở mức 50 người dùng, độ trễ p95 đạt **139 giây** và tỉ lệ lỗi lên tới **13%**, không đạt trải nghiệm chấp nhận được.

Nguyên nhân cốt lõi là mô hình ngôn ngữ `llama3.2` chạy cục bộ xử lý gần như tuần tự, trong khi mỗi câu hỏi cần hai lần gọi mô hình. Đây là nút thắt cổ chai quyết định năng lực của toàn hệ thống [3, 2].

Kết luận nhanh

Điểm bão hoà của cấu hình hiện tại là $\approx 5\text{--}10$ người dùng đồng thời, với thông lượng trần khoảng $0,7\text{--}0,75$ request/giây. Muốn phục vụ số lượng người dùng lớn hơn, bắt buộc phải gỡ nút thắt mô hình ngôn ngữ (tăng song song Ollama hoặc chuyển sang API LLM đám mây) và chạy máy chủ ứng dụng đa tiến trình.

2. Môi trường và phương pháp kiểm thử

2.1. Cấu hình môi trường

Bảng 1: Cấu hình môi trường kiểm thử

Hạng mục	Chi tiết
Công cụ kiểm thử	k6 v2.1.0 (Windows/amd64)
Endpoint	POST <code>/api/vanna/v2/chat_sse</code> (Server-Sent Events)
Mô hình AI	Ollama llama3.2 (chạy cục bộ, không tốn phí API)
Cơ sở dữ liệu	PostgreSQL — Neon, schema <code>qlsp_backup</code>
Máy chủ ứng dụng	Uvicorn (1 worker), FastAPI, Vanna 2.0
Loại truy vấn	Câu hỏi tiếng Việt (đếm và tổng hợp đơn giản)
Tiêu chí thành công	HTTP 200 và stream kết thúc bằng [DONE]
Timeout	mỗi request 180 giây
Baseline (1 người dùng)	$\approx 2,5$ giây/truy vấn

2.2. Cấu hình phần cứng

Toàn bộ hệ thống (ứng dụng, mô hình Ollama và công cụ k6) chạy trên cùng một máy trạm; cơ sở dữ liệu PostgreSQL đặt từ xa trên Neon (cloud). Đây là mô hình kiểm thử một máy (single-node), phù hợp cho giai đoạn PoC.

Bảng 2: Thông số phần cứng máy chạy kiểm thử

Thành phần	Thông số
CPU	AMD Ryzen 7 8845H (8 nhân / 16 luồng)
GPU	AMD Radeon 780M (đồ hoạ tích hợp)
RAM	16 GB (khoảng 13,8 GB khả dụng; phần còn lại chia sẻ cho GPU tích hợp)
Hệ điều hành	Windows 11 Home
Vị trí cơ sở dữ liệu	Neon (PostgreSQL đám mây, kết nối qua SSL)

Ghi chú: Do máy sử dụng **GPU tích hợp** (không có GPU rời), mô hình llama3.2 suy luận chủ yếu trên CPU. Đây là yếu tố phần cứng trực tiếp giới hạn tốc độ và mức độ song song của mô hình, và cần được tính đến khi diễn giải kết quả kiểm thử.

2.3. Đặc điểm kỹ thuật ảnh hưởng đến hiệu năng

- Mỗi câu hỏi kích hoạt **hai lần gọi mô hình ngôn ngữ**: một lần để sinh câu lệnh SQL, một lần để diễn giải kết quả trả về cho người dùng.
- Mỗi request sử dụng một **conversation_id** riêng biệt để tránh cơ chế khoá theo hội thoại (HTTP 409) của máy chủ, bảo đảm tải thực sự song song.
- Ollama cục bộ xử lý các yêu cầu suy luận gần như **tuần tự**, nên là điểm giới hạn thông lượng chính của hệ thống.

2.4. Kịch bản kiểm thử

Hai kịch bản tăng tải (**ramping-vus**) được thực hiện:

- Kịch bản A — Tăng dần:** 5 → 15 → 30 người dùng đồng thời, tổng thời gian ≈ 3 phút 35 giây. Mục tiêu quan sát diễn biến khi tải tăng từ thấp đến trung bình.
- Kịch bản B — Cao điểm 50:** tăng nhanh lên 50 người dùng và giữ trong 120 giây. Mục tiêu kiểm tra hành vi khi vượt xa ngưỡng bảo hoà.

Listing 1: Trích cấu hình kịch bản tăng tải trong k6

```
export const options = {
  scenarios: {
    ramping_users: {
      executor: 'ramping-vus',
      stages: [
        { duration: '20s', target: 5 }, // 5 người dùng
        { duration: '20s', target: 15 }, // tăng lên 15
```

```
{ duration: '60s', target: 30 }, // cao diem 30
],
},
},
};
```

2.5. Bộ câu hỏi kiểm thử

Mỗi người dùng ảo chọn ngẫu nhiên một câu hỏi trong tập câu hỏi nghiệp vụ tiếng Việt dưới đây. Các câu hỏi thuộc nhóm đếm và tổng hợp đơn giản nhằm cô lập chi phí suy luận của mô hình (tách khỏi chi phí truy vấn SQL phức tạp):

1. “Có bao nhiêu người dùng trong hệ thống?”
2. “Có bao nhiêu đơn vị trong bảng `units`?”
3. “Có bao nhiêu `spdv_tickets`?”
4. “Đếm số người dùng theo trạng thái.”
5. “Liệt kê 5 đơn vị đầu tiên theo tên.”
6. “Có bao nhiêu sự kiện trong `spdv_ticket_events`?”

3. Kết quả kiểm thử

3.1. Kết quả tổng quan

Bảng 3: So sánh kết quả tổng quan theo mức tải

Chỉ số	1 user (baseline)	30 user (A)	50 user (B)
Tổng số request	1	177	132
Thông lượng (req/giây)	0,40	0,75	0,70
Tỉ lệ thành công	100%	95,0%	87,0%
Số lỗi (rớt kết nối EOF)	0	9	17
Tỉ lệ lỗi HTTP	0%	5,0%	13,0%

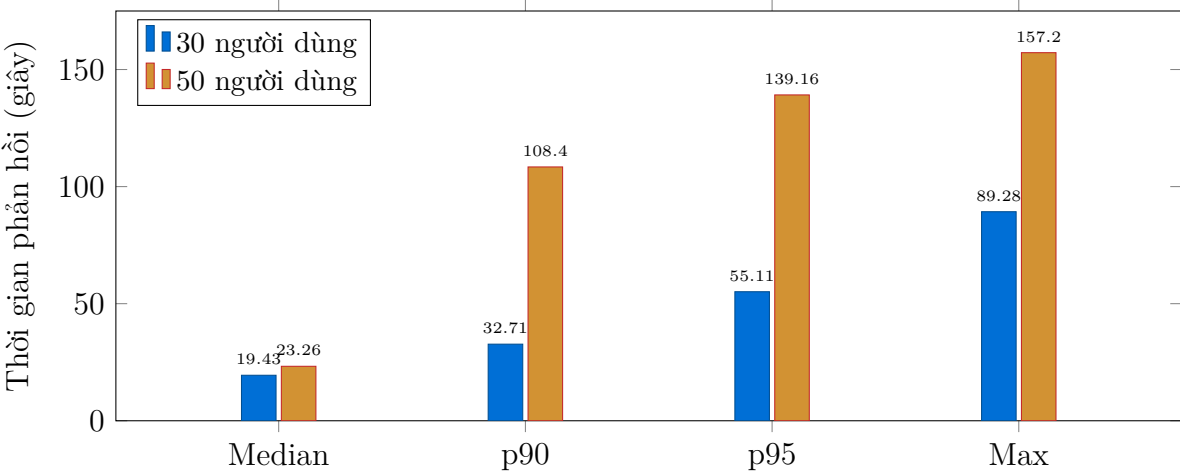
3.2. Phân bố thời gian phản hồi AI

Bảng 4: Phân bố thời gian phản hồi AI theo mức tải (đơn vị: giây)

Thời gian phản hồi	Baseline	30 user	50 user
Nhanh nhất (min)	–	0,59	1,34
Trung vị (median)	2,5	19,43	23,26
Trung bình (avg)	2,5	20,90	36,28
p90	–	32,71	108,40
p95	–	55,11	139,16
Chậm nhất (max)	2,5	89,28	157,20

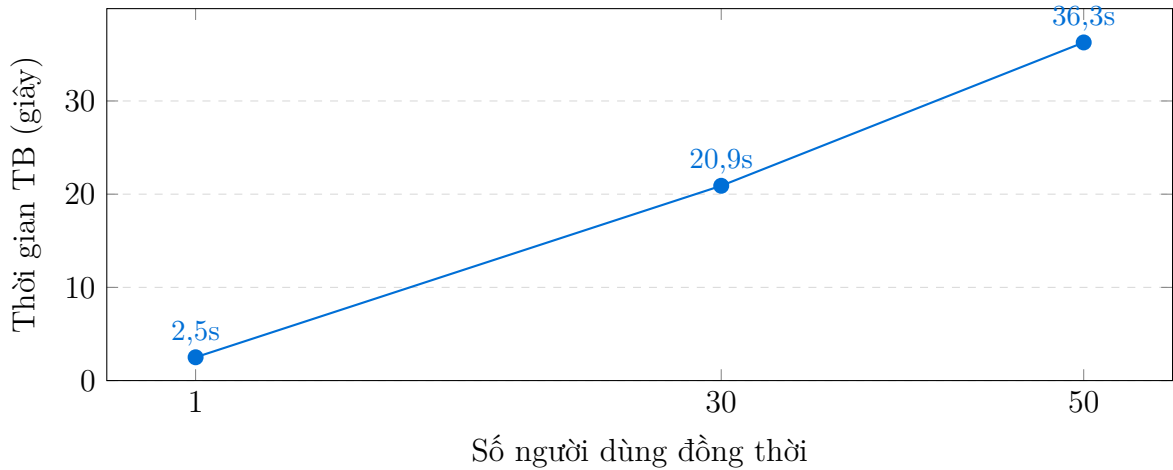
3.3. Biểu đồ trực quan

Hình 1 so sánh phân bố thời gian phản hồi giữa hai mức tải. Có thể thấy độ trễ tăng rất mạnh ở các percentile cao khi chuyển từ 30 lên 50 người dùng: p95 nhảy từ 55 giây lên 139 giây.



Hình 1: So sánh phân bố thời gian phản hồi AI ở mức 30 và 50 người dùng đồng thời.

Hình 2 cho thấy đặc trưng bão hoà: thời gian phản hồi trung bình tăng gần tuyến tính theo số người dùng, trong khi thông lượng không tăng — dấu hiệu điển hình của một hệ thống đã chạm trần xử lý.



Hình 2: Thời gian phản hồi trung bình tăng gần tuyến tính theo số người dùng (baseline $\rightarrow$ 30 $\rightarrow$ 50), trong khi thông lượng giữ nguyên $\approx 0,7$ req/giây.

3.4. Diễn biến theo mức tải

Bảng 5: Hành vi hệ thống quan sát theo từng giai đoạn tải

Giai đoạn	Số user	Hành vi hệ thống
Tải thấp	5	Ổn định, phản hồi nhanh, không có lỗi
Tải trung	15	Độ trễ tăng rõ, xuất hiện lỗi rút kết nối đầu tiên
Tải cao	30	Thông lượng chững lại, nhiều lỗi EOF, tỉ lệ lỗi 5%
Quá tải	50	Bão hoà hoàn toàn, p95 tới 139 giây, tỉ lệ lỗi 13%

4. Phân tích kết quả

4.1. Thông lượng đạt trần

Thông lượng gần như không đổi giữa 30 và 50 người dùng (0,75 so với 0,70 request/giây). Điều này chứng minh hệ thống đã **đạt trần xử lý** ở mức $\approx 0,7-0,75$ request/giây. Việc tăng thêm người dùng không làm tăng năng lực xử lý mà chỉ kéo dài hàng đợi, khiến độ trễ và tỉ lệ lỗi tăng vọt.

4.2. Giải thích bằng định luật Little

Định luật Little trong lý thuyết hàng đợi phát biểu:

$$L = \lambda \times W$$

trong đó L là số yêu cầu trung bình đang trong hệ thống (mức đồng thời), λ là thông lượng (request/giây) và W là thời gian phản hồi trung bình. Khi hệ thống đã bão hoà, thông lượng λ bị chặn trên bởi năng lực xử lý:

$$\lambda_{\max} \approx \frac{N_{\text{song song}}}{W_{\text{phục vụ}}}$$

Với thời gian phục vụ cơ sở $W_{\text{phục vụ}} \approx 2,5$ giây (đo ở baseline) và thông lượng trần quan sát được $\approx 0,75$ req/giây, có thể suy ra mức song song hiệu dụng $N_{\text{song song}} \approx 0,75 \times 2,5 \approx 1,9$. Con số này phù hợp với việc Ollama chỉ xử lý được khoảng **2 yêu cầu song song** trên cấu hình phần cứng hiện tại (CPU + GPU tích hợp). Khi số người dùng vượt xa mức này, mọi yêu cầu tăng thêm chỉ làm dài thời gian chờ W (theo đúng $L = \lambda \times W$ với λ cố định), chứ không nâng được thông lượng.

4.3. Nút thắt cổ chai: mô hình ngôn ngữ cục bộ

Nguyên nhân chính là **Ollama chạy cục bộ xử lý gần như tuần tự**. Với hai lần gọi mô hình cho mỗi câu hỏi, khi có 30–50 người dùng cùng lúc, các yêu cầu xếp hàng dài. Thời gian phản hồi tăng từ 2,5 giây (1 người dùng) lên trung vị ≈ 23 giây và p95 tới 139 giây (50 người dùng) — tức chậm gấp **hơn 50 lần** so với baseline.

4.4. Lỗi rớt kết nối

Toàn bộ lỗi thuộc dạng EOF: máy chủ chủ động đóng kết nối khi quá tải. Tỷ lệ lỗi tăng từ 5% (30 người dùng) lên 13% (50 người dùng). Nguyên nhân là sự kết hợp giữa hàng đợi Ollama quá dài, giới hạn số kết nối tới Neon, và Uvicorn chỉ chạy **một tiến trình worker** duy nhất.

Nhận định trọng yếu

Năng lực của hệ thống được quyết định bởi tốc độ suy luận của mô hình ngôn ngữ, không phải bởi cơ sở dữ liệu hay tầng ứng dụng. Do đó, mọi phương án nâng cao khả năng chịu tải phải ưu tiên gỡ nút thắt LLM trước tiên.

5. Đánh giá theo tiêu chí nghiệm thu (SLA)

Bảng 6 đối chiếu kết quả đo với các ngưỡng chất lượng dịch vụ đề xuất cho một hệ thống hỏi đáp tương tác. Ở mức 30 và 50 người dùng, hệ thống **không đạt** cả ba tiêu chí; các ngưỡng này chỉ được thỏa mãn khi số người dùng đồng thời ở mức thấp (≤ 10).

Bảng 6: Đối chiếu kết quả với tiêu chí nghiệm thu đề xuất

Tiêu chí	Ngưỡng mục tiêu	30 user	50 user
Độ trễ p95	< 10 giây	Không đạt	Không đạt
Tỷ lệ thành công	$\geq 99\%$	Không đạt	Không đạt
Tỷ lệ lỗi	$\leq 1\%$	Không đạt	Không đạt

Sức chứa an toàn ước tính

Với ngưỡng p95 dưới 10 giây và tỷ lệ lỗi dưới 1%, cấu hình hiện tại chỉ nên phục vụ tối đa **5–10 người dùng đồng thời**. Để phục vụ nhiều hơn, cần áp dụng các khuyến nghị ở Mục 7 trước khi đưa vào sử dụng thực tế.

6. Giới hạn của kiểm thử

Kết quả trong báo cáo cần được diễn giải trong phạm vi các giới hạn sau:

- **Mô hình một máy:** ứng dụng, mô hình Ollama và công cụ k6 chạy trên cùng một máy, nên có sự cạnh tranh tài nguyên CPU giữa bên tạo tải và bên xử lý.
- **Phần cứng tiêu dùng:** máy dùng GPU tích hợp, không đại diện cho cấu hình máy chủ production có GPU rời; kết quả tuyệt đối sẽ khác trên hạ tầng mạnh hơn.
- **Câu hỏi đơn giản:** bộ câu hỏi thiên về đếm/tổng hợp; câu hỏi phức tạp (nhiều join, cửa sổ thời gian) sẽ làm tăng cả chi phí SQL lẫn suy luận.
- **Độ dài kiểm thử:** mỗi kịch bản kéo dài vài phút, chưa đánh giá độ ổn định dài hạn (rò rỉ bộ nhớ, suy giảm hiệu năng theo thời gian).
- **Cơ sở dữ liệu đám mây:** độ trễ mạng tới Neon có thể dao động theo thời điểm, ảnh hưởng nhỏ đến kết quả.

7. Kết luận

Điểm bão hoà của hệ thống hiện tại $\approx$ 5–10 người dùng đồng thời.

Dưới 10 người dùng: phản hồi nhanh, gần như không lỗi. Từ 15 người dùng: độ trễ tăng phi tuyến, bắt đầu rớt kết nối. Ở 30–50 người dùng: p95 đạt 55–139 giây và 5–13% request thất bại — không đạt trải nghiệm chấp nhận được cho người dùng thực.

Kết quả khẳng định rằng để phục vụ số lượng người dùng lớn hơn, việc tối ưu tăng ứng dụng là chưa đủ; cần đầu tư vào năng lực suy luận của mô hình ngôn ngữ và kiến trúc phục vụ đồng thời.

8. Khuyến nghị cải thiện

Bảng 7: Các khuyến nghị nâng cao khả năng chịu tải

Ưu tiên	Giải pháp	Kỳ vọng
Cao	Chạy Unicorn nhiều worker (<code>--workers 4</code>) hoặc đặt sau Gunicorn	Giảm rớt kết nối, tận dụng đa nhân
Cao	Tăng song song Ollama (<code>OLLAMA_NUM_PARALLEL</code>) hoặc chuyển sang API đám mây (Anthropic/OpenAI) cho tải cao	Gỡ nút thắt LLM — yếu tố quyết định
Trung bình	Cache câu hỏi và câu lệnh SQL lặp lại	Giảm số lần gọi mô hình
Trung bình	Cấu hình connection pool tới Neon hợp lý, tăng timeout	Giảm lỗi EOF

Ưu tiên	Giải pháp	Kỳ vọng
Thấp	Thêm hàng đợi và thông báo “đang xử lý” cho người dùng	Cải thiện trải nghiệm khi tải cao

Người thực hiện
(Đã lập báo cáo)

Lê Đức Anh

Phụ lục A. Kịch bản k6 (rút gọn)

Listing 2: Logic phát tải và đo lường của mỗi người dùng ảo

```
import http from 'k6/http';
import { check } from 'k6';
import { Trend, Rate, Counter } from 'k6/metrics';

const aiResponseTime = new Trend('ai_response_time', true);
const aiSuccessRate = new Rate('ai_success_rate');
const aiErrors = new Counter('ai_errors');

export default function () {
  // conversation_id duy nhất -> tránh khoa 409 theo hỏi thoại
  const uid = `lt-${__VU}-${__ITER}-${Date.now()}-${Math.random()}`;
  const question = QUESTIONS[Math.floor(Math.random() * QUESTIONS.length)];

  const res = http.post(ENDPOINT,
    JSON.stringify({ message: question, conversation_id: uid, request_id: uid }),
    { headers: { 'Content-Type': 'application/json' }, timeout: '180s' });

  aiResponseTime.add(res.timings.duration);
  const ok = res.status === 200 && String(res.body).includes('data: [DONE]');
  aiSuccessRate.add(ok);
  if (!ok) aiErrors.add(1);

  check(res, {
    'HTTP 200': (r) => r.status === 200,
    'stream hoàn tất': (r) => String(r.body).includes('data: [DONE]'),
  });
}
```

Mã nguồn đầy đủ và số liệu thô có trong thư mục loadtest/ của kho mã: ai_load_test.js, ai_load_test_50.js, k6_summary.json, k6_summary_50.json.

Tài liệu tham khảo

- [1] Grafana Labs, *k6 — Load testing tool for engineering teams*, tài liệu công cụ kiểm thử tải mã nguồn mở, phiên bản 2.1.0. <https://k6.io/docs/>. Truy cập ngày 04/08/2026.
- [2] Vanna AI, *Vanna 2.0: Turn Questions into Data Insights*, GitHub repository [vanna-ai/vanna](https://github.com/vanna-ai/vanna), phiên bản mã nguồn tham chiếu 2.0.2. <https://github.com/vanna-ai/vanna>. Truy cập ngày 04/08/2026.
- [3] Ollama, *Ollama — Run large language models locally*, tài liệu cấu hình song song (OLLAMA_NUM_PARALLEL) và phục vụ mô hình. <https://github.com/ollama/ollama>. Truy cập ngày 04/08/2026.
- [4] Neon, *Connection pooling*, tài liệu kết nối PostgreSQL và pooled connection cho ứng dụng. <https://neon.com/docs/connect/connection-pooling>. Truy cập ngày 04/08/2026.